



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

NEURONOWY KODEK MOWY O ULTRA-NISKIEJ SZYBKOŚCI TRANSMISJI DANYCH DO PRZETWARZANIA BRZEGOWEGO

Jakub Aszyk, 151841
Kosma Gąsiorowski, 165575 ???

Promotor
dr inż. Mariusz Nowak

POZNAŃ 2026

Spis treści

1	Wstęp	1
1.1	Wprowadzenie do problematyki kompresji dźwięku przy bardzo niskich przepływnościach	1
1.2	Motywacja do podjęcia tematu	1
1.3	Cel i zakres pracy	1
1.4	Podział prac	1
2	Podstawy teoretyczne	3
2.1	Analiza literatury zagadnienia	3
2.1.1	Zakres przeglądu i kryteria porównania	3
2.1.2	Kodeki parametryczne i hybrydowe	4
2.1.3	Neuronowe kodeki end-to-end i reprezentacje dyskretne	4
2.2	Aktualny stan technologii kodowania mowy i dźwięku poniżej 1 kbps	5
2.2.1	Rozwiązania rzeczywiście pracujące w zakresie subkilobitowym	5
2.2.2	Granica między kodekami subkilobitowymi a neuronowymi	6
2.2.3	Podejścia semantyczne i ograniczenia urządzeń brzegowych	6
2.3	Charakterystyka sygnału mowy i dźwięku	7
2.4	Podstawy cyfrowej reprezentacji sygnału audio	7
2.5	Przepływność bitowa i jej wpływ na jakość dźwięku	7
2.6	Podstawy neuronowej kompresji sygnału	7
2.6.1	Sieci neuronowe jako modele przetwarzania sygnałów	7
2.6.2	Autoenkodery i wąskie gardło informacyjne	7
2.6.3	Konwolucje jednowymiarowe w analizie sygnału audio	8
2.6.4	Reprezentacja latentna i redukcja czasowa	8
2.6.5	Kwantyzacja wektorowa i rezydualna kwantyzacja wektorowa	9
2.6.6	Funkcje aktywacji, normalizacja i regularyzacja	10
2.6.7	Funkcje strat stosowane w rekonstrukcji audio	10
2.7	Metody kodowania dźwięku przy bardzo niskich przepływnościach	11
2.8	Metryki oceny jakości rekonstrukcji dźwięku	11
3	Metoda i implementacja systemu	12
3.1	Założenia projektowe proponowanego rozwiązania	12
3.2	Dobór reprezentacji sygnału wejściowego	13
3.2.1	Fala mowy jako wejście modelu	13
3.2.2	Próbkowanie, segmentacja i normalizacja	14
3.2.3	Reprezentacje pomocnicze w funkcji straty	14
3.3	Architektura kodera i dekodera	15

3.3.1	Tor kodera i reprezentacja latentna	15
3.3.2	Kontekst latentny i kwantyzacja RVQ	16
3.3.3	Dekoder konwolucyjny i post-filtr	17
3.4	Proces kompresji i rekonstrukcji sygnału	18
3.4.1	Przygotowanie i kodowanie segmentu	18
3.4.2	Przepływność nominalna i pakowanie indeksów	19
3.4.3	Rekonstrukcja po stronie dekodera	20
3.5	Implementacja systemu oraz wykorzystane technologie	21
3.5.1	Układ repozytorium i backendy	21
3.5.2	Artefakty treningu i inferencja	22
3.5.3	Środowisko uruchomieniowe i testy	23
4	Eksperymenty i analiza wyników	24
4.1	Opis zbioru danych wykorzystanego w badaniach	24
4.2	Konfiguracja eksperymentów	24
4.3	Wpływ przepływności bitowej na jakość rekonstrukcji	24
4.4	Wpływ parametrów modelu na skuteczność kompresji	24
4.5	Ocena jakości odtworzonego dźwięku	24
4.6	Dyskusja wyników	24
5	Zakończenie	25
5.1	Wnioski z przeprowadzonych badań	25
5.2	Realizacja celu i zakresu pracy	25
5.3	Ograniczenia techniczne zaproponowanego rozwiązania	25
5.4	Kierunki dalszego rozwoju	25
	Literatura	26
	Literatura	26
A	Dodatki	29
A.1	Repozytorium projektu	29
A.2	Konfiguracje eksperymentów	29
A.3	Dodatkowe wyniki i przykłady rekonstrukcji	29

Rozdział 1

Wstęp

1.1 Wprowadzenie do problematyki kompresji dźwięku przy bardzo niskich przepływnościach

1.2 Motywacja do podjęcia tematu

1.3 Cel i zakres pracy

1.4 Podział prac

Praca miała charakter zespołowy, dlatego podział zadań należy rozumieć jako podział odpowiedzialności merytorycznej za poszczególne obszary projektu, a nie jako prosty podział według historii zmian w repozytorium. Część decyzji projektowych, implementacyjnych i eksperymentalnych była podejmowana wspólnie podczas pracy przy jednym stanowisku. Było to szczególnie istotne przy iteracyjnym strojeniu architektury kodeka oraz interpretacji wyników odsłuchowych i metrycznych.

Do zadań wspólnych należał przegląd literatury dotyczącej wykorzystania sieci neuronowych do budowy kodeków mowy, w szczególności metod opartych na autoenkoderach, kwantyzacji wektorowej i uczeniu reprezentacji dyskretnych. Wspólnie opracowano także koncepcję kodeka mowy oraz wymagania projektowe obejmujące docelową przepływność bitową, jakość rekonstrukcji, opóźnienie przetwarzania i warianty działania systemu. Na tej podstawie przygotowano ogólną architekturę rozwiązania, obejmującą koder sygnału, warstwę reprezentacji latentnej, moduł kwantyzacji oraz dekodek fali dźwiękowej. Wspólnie określono również protokół ewaluacji, obejmujący zarówno miary obiektywne, jak i analizę artefaktów rekonstrukcji, a końcowe wnioski sformułowano na podstawie wyników uzyskanych przez obie osoby.

Jakub Aszyk odpowiadał za część związaną z koderem wejściowym, przygotowaniem danych oraz wdrożeniem systemu na platformie brzegowej. W ramach implementacji opracował konwulcyjny tor kodera, którego zadaniem było przekształcenie surowej fali mowy do krótszej sekwencji wektorów latentnych przekazywanych następnie do kwantyzacji. Obejmowało to dobór liczby etapów redukcji czasowej, szerokości kanałów, wymiaru reprezentacji latentnej oraz sposobu przygotowania wyjścia kodera pod moduł RVQ. Część ta była istotna, ponieważ bezpośrednio wpływała na nominalną przepływność bitową, pojemność informacyjną reprezentacji i koszt dalszego dekodowania. Jakub przygotował również pipeline danych dla korpusów mowy, mechanizmy konfiguracji eksperymentów oraz narzędzia do logowania przebiegów treningu i porządkowania artefaktów, takich jak checkpointy, spektrogramy i przykłady rekonstruowanego dźwięku. W części związanej z przetwarzaniem brzegowym odpowiadał za analizę czasu inferencji, wykorzystania pamięci i stabil-

ności działania modelu, a także za przygotowanie wersji eksportowalnej, konwersję do TensorFlow Lite oraz testy zgodności wyników względem implementacji referencyjnej.

Kosma Gąsiorowski odpowiadał za część modelową związaną z rekonstrukcją i kwantyzacją oraz za zasadniczą część badań jakościowych. W ramach implementacji rozwijał moduł rezydualnej kwantyzacji wektorowej, dekodery rekonstruujące przebieg czasowy oraz warianty poprawiające jakość odtwarzania, takie jak bloki temporalne w przestrzeni latentnej, elementy samouwagi, dodatkowe głowice dekodera, aktywacje okresowe oraz moduł rekonstrukcji wysokoczęstotliwościowych składowych sygnału. Kosma przygotował również funkcje kosztu wykorzystywane podczas uczenia, w tym straty czasowe, wieloskalowe straty STFT, straty melowe, składniki stabilizujące kwantyzację oraz kary związane z wykorzystaniem słowników kodowych. W części eksperymentalnej odpowiadał za strojenie hiperparametrów, porównania wariantów architektury, analizę ablacyjną, ocenę jakości odtworzonego dźwięku oraz interpretację błędów na podstawie metryk, spektrogramów i testów porównawczych. Do jego zakresu należało także porządkowanie wersji eksperymentów w repozytorium oraz opracowanie dokumentacji uzyskanych wyników i badań analitycznych.

Rozdział 2

Podstawy teoretyczne

Rozdział przedstawia podstawy teoretyczne potrzebne do uzasadnienia projektu neuronowego kodeka mowy o bardzo niskiej przepływności bitowej. Zagadnienie to znajduje się na przecięciu klasycznego kodowania mowy, przetwarzania sygnałów, uczenia reprezentacji dyskretnych oraz optymalizacji modeli pod urządzenia o ograniczonych zasobach. Z tego względu w pierwszej kolejności omówiono literaturę dotyczącą kodeków parametrycznych, hybrydowych i neuronowych, a następnie scharakteryzowano aktualny stan technologii w zakresie przepływności poniżej 1 kbps [34, 39, 45, 5]. Dalsze części rozdziału stanowią podstawę do opisu zaproponowanego rozwiązania, ponieważ wyjaśniają właściwości sygnału mowy, cyfrową reprezentację audio, znaczenie przepływności bitowej, podstawy neuronowej kompresji sygnału oraz metryki stosowane do oceny jakości rekonstrukcji.

Warto zaznaczyć, że literatura dotycząca kodeków mowy o ultra-niskiej przepływności nie tworzy jednorodnego pola badawczego. Część prac koncentruje się na klasycznym modelowaniu źródła mowy, część na neuronowym generowaniu sygnału z niewielkiej liczby parametrów, a część na pełnych kodekach uczonych end-to-end. Porównanie tych podejść wymaga ostrożności, ponieważ poszczególne prace stosują różne zbiory danych, różne procedury testów odsłuchowych oraz różne metryki obiektywne. W pracy przyjęto więc perspektywę projektową: istotne są nie tylko raportowane wartości MOS, PESQ lub ViSQOL, ale także możliwość uzyskania niskiej przepływności, stabilność kwantyzacji oraz koszt inferencji na platformie brzegowej.

2.1 Analiza literatury zagadnienia

2.1.1 Zakres przeglądu i kryteria porównania

Przegląd literatury w niniejszej pracy obejmuje publikacje dotyczące ultra-niskobitowej kompresji mowy, neuronowych kodeków audio oraz wdrażania modeli przetwarzania sygnałów na urządzeniach brzegowych. Uwzględniono przede wszystkim prace z lat 2015–2024, ze szczególnym naciskiem na rozwiązania działające przy przepływnościach rzędu pojedynczych kilobitów na sekundę lub niższych. Analizie poddano architekturę kodeka, technikę kwantyzacji, sposób parametryzacji sygnału mowy, metryki jakości oraz wymagania zasobowe. Takie kryteria są istotne, ponieważ w projektowanym systemie sama deklaracja niskiego bitrate’u nie wystarcza; konieczne jest także określenie, czy rozwiązanie może działać w warunkach ograniczonej pamięci, ograniczonej mocy obliczeniowej i małego dopuszczalnego opóźnienia.

Na poziomie technologicznym przegląd wskazuje trzy główne klasy rozwiązań. Pierwszą z nich są kodeki parametryczne, które opisują mowę za pomocą niewielkiego zestawu parametrów, takich jak współczynniki predykcji liniowej, częstotliwość podstawowa i energia. Drugą grupę stanowią

kodeki hybrydowe, łączące klasyczne modelowanie mowy z lekkim komponentem neuronowym. Trzecią grupą są kodeki neuronowe end-to-end, w których koder, kwantyzator i dekodery są uczone jako jeden system [34, 39, 45, 5]. Każda z tych rodzin rozwiązań ma inne ograniczenia: kodeki parametryczne są zwykle bardzo lekkie, ale mniej naturalne percepcyjnie; kodeki hybrydowe oferują kompromis między jakością i kosztem; natomiast rozwiązania end-to-end mogą uzyskiwać wysoką jakość, lecz często wymagają większych modeli i kosztowniejszego treningu.

2.1.2 Kodeki parametryczne i hybrydowe

Najniższe przepływności osiągają rozwiązania parametryczne, w których nie rekonstruuje się bezpośrednio pełnego przebiegu audio, lecz przesyła parametry modelu mowy. Przykładem takiego podejścia jest Codec 2, rozwijany jako otwarty kodek niskobitowy i raportowany w wariantach rzędu 700 bit/s oraz 450 bit/s [34, 35]. Tak niska przepływność jest możliwa dzięki silnemu wykorzystaniu założeń o strukturze mowy, jednak wiąże się z ograniczoną naturalnością rekonstrukcji. Zbliżoną rodzinę rozwiązań reprezentuje standard MELPe, opisany w STANAG 4591, który działa w trybach 600, 1200 i 2400 bit/s [24]. Warto zauważyć, że te rozwiązania pozostają ważnym punktem odniesienia dla pracy, ponieważ pokazują dolną granicę możliwej przepływności, ale nie odpowiadają bezpośrednio na pytanie, jak wykorzystać sieci neuronowe do poprawy jakości odtwarzanego sygnału.

Innym nurtem w tej grupie są metody oparte na predykcji liniowej i efektywnym kodowaniu parametrów. Backstrom i Magi zaproponowali rzadką predykcję liniową jako sposób poprawy efektywności kodowania przy niskich przepływnościach [4]. W takim podejściu nadal wykorzystuje się klasyczny opis sygnału mowy, ale modyfikuje się sposób reprezentacji i kodowania parametrów. Rozwiązania tego typu są istotne dla projektowanego systemu, ponieważ wskazują, że ograniczenie liczby przesyłanych informacji może być osiągane nie tylko przez zmniejszenie modelu, lecz także przez odpowiedni dobór reprezentacji wejściowej i słownika kodowego.

Kodeki hybrydowe próbują połączyć zalety klasycznego modelowania mowy z większą elastycznością modeli neuronowych. Przykładem jest LPCNet, w którym predykcja liniowa jest wykorzystywana do ograniczenia zadania stojącego przed siecią neuronową, a sam model odpowiada za generowanie lub poprawę sygnału mowy [39]. Valin, Bütche i Vos pokazali również, że rozwiązania oparte na lekkich strukturach rekurencyjnych mogą być optymalizowane pod procesory ARM i pracę w czasie rzeczywistym [38]. Uzupełnieniem tego kierunku jest praca Zhu i Valina, w której opisano optymalizację LPCNet dla urządzeń embedded [47]. Znaczenie tych prac polega na tym, że nie traktują modelu neuronowego jako pełnego zamiennika klasycznego kodeka, lecz jako komponent poprawiający jakość przy zachowaniu niskiej złożoności.

2.1.3 Neuronowe kodeki end-to-end i reprezentacje dyskretne

Rozwój neuronowych kodeków audio znacząco zmienił sposób projektowania systemów kompresji. SoundStream zaproponował architekturę end-to-end obejmującą koder, rezydualną kwantyzację wektorową oraz dekodery generujące sygnał audio [45]. Podobny kierunek rozwinięto w EnCodec, gdzie zastosowano wysokiej jakości neuronową kompresję audio i wielostopniową kwantyzację latentów [5]. Prace te są szczególnie istotne dla niniejszej pracy, ponieważ pokazują, że reprezentacja latentna i RVQ mogą tworzyć efektywny mechanizm zamiany sygnału audio na strumień dyskretnych indeksów. Jednocześnie należy zauważyć, że raportowane warianty tych kodeków zwykle działają przy przepływnościach wyższych niż skrajnie niskie zakresy poniżej 1 kbps, a ich koszt obliczeniowy może być problematyczny dla prostych urządzeń brzegowych.

W literaturze występują również próby poprawy kwantyzacji i jakości rekonstrukcji przez modyfikację struktury słowników. HiFi-Codec wprowadza grupową rezydualną kwantyzację wektorową, która ma zwiększać wierność rekonstrukcji przy kontrolowanej liczbie kodów [44]. Z kolei prace oparte na dyskretnych reprezentacjach samonadzorowanych pokazują, że sygnał mowy może być rekonstruowany z tokenów lub cech oderwanych częściowo od pełnej postaci fali [28]. SpeechTokenizer rozwija ten kierunek, łącząc reprezentacje semantyczne i akustyczne w kontekście modeli mowy [46]. Z perspektywy kodeka o ultra-niskiej przepływności podejścia semantyczne są obiecujące, ponieważ mogą redukować liczbę przesyłanych informacji do treści fonetycznej lub lingwistycznej. Ich ograniczeniem pozostaje jednak koszt ekstrakcji cech i ryzyko utraty informacji potrzebnych do naturalnej rekonstrukcji barwy, prozodii i szczegółów artykulacyjnych.

Osobną grupę stanowią prace, które nie są bezpośrednio pełnymi kodekami mowy, lecz wywarły wpływ na projektowanie neuronowych generatorów audio. WaveNet pokazał, że autoregresyjne modele fali mogą generować wysokiej jakości sygnał audio, choć kosztem znacznej złożoności obliczeniowej [40]. Kleijn i inni zastosowali podejście WaveNet do niskobitowego kodowania mowy [19]. Kankanahalli zaproponował natomiast kodowanie mowy optymalizowane end-to-end z wykorzystaniem głębokich sieci neuronowych [17]. W kontekście urządzeń brzegowych istotna jest również praca Morrisona, Xu i Bryana, dotycząca efektywnych neuronowych kodeków audio dla strumieniowania o małym opóźnieniu [23]. Uzupełniającym punktem odniesienia są metody sinusoidalne, w których sygnał reprezentowany jest przez składowe harmoniczne i szumowe [8]. Łącznie prace te pokazują, że wysoka jakość neuronowej rekonstrukcji jest możliwa, ale wymaga szczególnej kontroli złożoności, jeżeli system ma działać na brzegu sieci.

2.2 Aktualny stan technologii kodowania mowy i dźwięku poniżej 1 kbps

2.2.1 Rozwiązania rzeczywiście pracujące w zakresie subkilobitowym

W zakresie poniżej 1 kbps dominują obecnie przede wszystkim rozwiązania parametryczne. Codec 2 w wariancie 700 bit/s oraz eksperymentalny wariant 450 bit/s pokazują, że mowa może być przesyłana przy bardzo małej liczbie bitów, jeśli koder wykorzystuje silne założenia o strukturze sygnału [34, 35]. Podobny charakter ma najniższy tryb MELPe pracujący przy 600 bit/s [24]. Rozwiązania te pozostają praktyczne pod względem zasobów, jednak ich jakość percepcyjna jest ograniczona. W praktyce oznacza to, że bardzo niski bitrate uzyskuje się kosztem naturalności, szerokości pasma, szczegółów artykulacyjnych i odporności na nietypowe głosy lub warunki akustyczne.

Warto zaznaczyć, że subkilobitowe kodowanie mowy rzadko polega na zachowaniu pełnej informacji akustycznej. Częściej jest to kodowanie informacji wystarczającej do zrozumienia wypowiedzi, a nie do wiernej rekonstrukcji naturalnego brzmienia. Z tego powodu klasyczne kodeki parametryczne są nadal konkurencyjne w najniższych zakresach bitrate'u, mimo że nie odpowiadają współczesnym oczekiwaniom dotyczącym jakości dźwięku. W kontekście niniejszej pracy oznacza to, że neuronowy kodek poniżej 1 kbps musi rozwiązać trudniejszy problem niż kodeki pracujące przy kilku kilobitach na sekundę: powinien utrzymać zrozumiałość mowy, a jednocześnie ograniczyć artefakty typowe dla nadmiernie uproszczonego modelu źródła.

2.2.2 Granica między kodekami subkilobitowymi a neuronowymi

Neuronowe kodeki end-to-end, takie jak SoundStream i EnCodec, wyznaczają obecnie ważny kierunek technologiczny, ale ich typowe konfiguracje nie są projektowane przede wszystkim dla zakresu poniżej 1 kbps [45, 5]. EnCodec raportuje m.in. tryb 1,5 kbps, co jest już bliskie zakresowi ultra-niskiemu, lecz nadal przekracza próg 1 kbps i wymaga większych zasobów niż klasyczne kodeki parametryczne [5]. SoundStream i HiFi-Codec pokazują natomiast, że RVQ oraz warianty grupowej kwantyzacji są skuteczne przy budowie dyskretnych reprezentacji audio, ale przeniesienie tych metod do zakresu subkilobitowego wymaga znacznego zmniejszenia liczby przesyłanych indeksów lub rozmiarów słowników [45, 44].

Istotną luką technologiczną pozostaje więc zakres między klasycznymi kodekami parametrycznymi a dużymi neuronowymi kodekami audio. Kodeki parametryczne działają poniżej 1 kbps, ale oferują ograniczoną jakość. Kodeki neuronowe oferują wyższą jakość, ale zwykle przy większym bitrate i wyższym koszcie obliczeniowym. Podejścia hybrydowe, takie jak LPCNet, stanowią próbę wypełnienia tej luki, ponieważ wykorzystują parametryczny opis mowy do ograniczenia zadania stawianego sieci neuronowej [39, 47]. Wciąż jednak większość praktycznych wariantów hybrydowych raportowana jest dla przepływności około 1,6 kbps lub wyższej, co oznacza, że zejście poniżej 1 kbps wymaga dalszej redukcji informacji przesyłanej przez kanał.

2.2.3 Podejścia semantyczne i ograniczenia urządzeń brzegowych

Jednym z najbardziej interesujących kierunków dla przyszłych kodeków subkilobitowych są reprezentacje semantyczne. SpeechTokenizer oraz prace dotyczące rekonstrukcji mowy z dyskretnych reprezentacji samonadzorowanych sugerują, że część informacji akustycznej można zastąpić tokenami opisującymi treść lub strukturę fonetyczną wypowiedzi [46, 28]. Takie podejście może być korzystne przy bardzo niskiej przepływności, ponieważ liczba tokenów semantycznych może być mniejsza niż liczba informacji potrzebnych do pełnej rekonstrukcji fali. Jednocześnie metoda ta wprowadza nowe ograniczenia: model ekstrakcji reprezentacji semantycznej bywa duży, a rekonstrukcja naturalnego brzmienia wymaga dodatkowych informacji akustycznych, których nie da się łatwo zmieścić w bardzo małym budżecie bitowym.

W przypadku przetwarzania brzegowego kluczowe znaczenie mają opóźnienie, pamięć i możliwość pracy bez akceleratora GPU. Prace dotyczące optymalizacji LPCNet i modeli mowy na procesorach ARM pokazują, że lekkie architektury oraz wykorzystanie instrukcji SIMD/NEON mogą istotnie zmniejszyć koszt inferencji [38, 47]. Z kolei badania nad efektywnymi neuronowymi kodekami audio dla strumieniowania o małym opóźnieniu wskazują, że architektura modelu musi być projektowana razem z wymaganiami wdrożeniowymi, a nie dopiero optymalizowana po zakończeniu treningu [23]. Dla projektowanego kodeka oznacza to konieczność kontrolowania nie tylko jakości rekonstrukcji, ale także długości okna kodowania, liczby operacji w koderze i dekodерze oraz rozmiaru pamięci potrzebnej do przechowywania parametrów modelu.

Podsumowując, aktualny stan technologii pokazuje, że poniżej 1 kbps najpewniejszym punktem odniesienia pozostają kodeki parametryczne, natomiast rozwiązania neuronowe nadal wymagają kompromisów. Najbardziej uzasadnionym kierunkiem dla pracy jest więc projekt inspirowany neuronowymi kodekami end-to-end, ale silnie ograniczony pod względem przepływności i złożoności. W praktyce oznacza to wykorzystanie kodera generującego bardzo zwartą reprezentację latentną, kwantyzacji RVQ lub jej uproszczonego wariantu oraz dekodera zaprojektowanego tak, aby przy małej liczbie bitów rekonstruował przede wszystkim informację istotną percepcyjnie dla sygnału mowy.

2.3 Charakterystyka sygnału mowy i dźwięku

2.4 Podstawy cyfrowej reprezentacji sygnału audio

2.5 Przepływność bitowa i jej wpływ na jakość dźwięku

2.6 Podstawy neuronowej kompresji sygnału

Neuronowa kompresja sygnału audio polega na zastąpieniu części ręcznie projektowanych etapów kodeka modelem uczonym na danych. W klasycznych systemach kodowania znaczną część struktury narzuca projektant: wybiera transformację, parametry modelu mowy, sposób kwantyzacji i reguły rekonstrukcji. W podejściu neuronowym część tych decyzji zostaje przeniesiona do procesu uczenia, w którym model optymalizuje reprezentację pośrednią tak, aby umożliwiała rekonstrukcję sygnału przy ograniczonej liczbie przesyłanych informacji [10, 45, 5]. Nie oznacza to jednak rezygnacji z wiedzy dziedzinowej. W praktyce architektura modelu nadal musi narzucać odpowiednie ograniczenia: małą częstotliwość ramek latentnych, skończony słownik kodów, ograniczoną liczbę warstw oraz możliwość uruchomienia na urządzeniu o małych zasobach.

Z punktu widzenia niniejszej pracy najważniejsze są te elementy uczenia głębokiego, które bezpośrednio wpływają na konstrukcję kodeka: autoenkoderowy podział na koder i dekodek, konwulucyjne przetwarzanie przebiegu czasowego, zwarta reprezentacja latentna, kwantyzacja wektorowa oraz dobór funkcji strat. Omówienie tych zagadnień ma charakter użytkowy. Celem nie jest pełny opis teorii sieci neuronowych, lecz wskazanie, dlaczego dana klasa mechanizmów jest przydatna przy kompresji mowy poniżej 1 kbps.

2.6.1 Sieci neuronowe jako modele przetwarzania sygnałów

Podstawową jednostką sieci neuronowej jest neuron obliczeniowy, który tworzy kombinację liniową wejść, dodaje przesunięcie, a następnie stosuje funkcję nieliniową. Historycznie jednym z pierwszych formalnych modeli tego typu był perceptron Rosenblatta [33]. Współczesne sieci głębokie składają się jednak z wielu warstw takich przekształceń, dzięki czemu mogą modelować zależności nieliniowe znacznie bardziej złożone niż pojedynczy klasyfikator liniowy [10]. Dla sygnałów audio jest to istotne, ponieważ zależność między przebiegiem czasowym, strukturą harmoniczną, artykulacją i percepcyjną jakością dźwięku nie jest prosta ani lokalnie liniowa.

Uczenie sieci polega na minimalizacji funkcji kosztu względem parametrów modelu. W praktyce wykorzystuje się algorytm propagacji wstecznej błędu, który umożliwia obliczanie gradientów dla kolejnych warstw modelu [36]. Parametry aktualizowane są następnie przez metodę optymalizacji, często w wariacie adaptacyjnym, takim jak Adam [18]. W projektach implementacyjnych istotne są również biblioteki automatycznego różniczkowania, ponieważ pozwalają definiować złożone modele i funkcje strat bez ręcznego wyprowadzania pochodnych; przykładem takiego środowiska jest PyTorch [27]. Z punktu widzenia kodeka mowy sam mechanizm uczenia jest jednak tylko narzędziem. Kluczowe pozostaje to, jakie ograniczenia zostaną narzucone reprezentacji pośredniej i czy model nauczy się przenosić przez wąski kanał informacje ważne percepcyjnie.

2.6.2 Autoenkodery i wąskie gardło informacyjne

Autoenkoder jest modelem złożonym z dwóch głównych części: koder oraz dekodek. Koder przekształca sygnał wejściowy x do reprezentacji pośredniej z , natomiast dekodek rekonstruuje sygnał $\hat{x}$ na podstawie tej reprezentacji:

$$z = f_{\theta}(x), \quad \hat{x} = g_{\phi}(z). \quad (2.1)$$

Parametry θ i ϕ są dobierane tak, aby zminimalizować błąd rekonstrukcji. Autoenkodery były szeroko wykorzystywane jako narzędzie uczenia reprezentacji i redukcji wymiarowości danych [14, 10]. W kompresji ich rola jest szczególnie naturalna: koder odpowiada za wyodrębnienie informacji istotnej z punktu widzenia rekonstrukcji, a dekodery za odtworzenie sygnału z ograniczonego opisu.

Samo zastosowanie autoenkodera nie wystarcza jeszcze do zbudowania kodeka. Jeżeli reprezentacja z jest ciągła i ma dużą pojemność, może przechowywać bardzo dokładny opis wejścia, ale nie tworzy bezpośrednio skończonego strumienia bitów. Dlatego w systemach kompresji wprowadza się wąskie gardło informacyjne: zmniejsza się częstotliwość ramek latentnych, ogranicza wymiar wektorów, stosuje kwantyzację albo łączy te mechanizmy. W neuronowych kodekach audio, takich jak SoundStream i EnCodec, autoenkoder stanowi podstawę architektury, ale dopiero kwantyzacja wektorowa zamienia reprezentację latentną na dyskretny strumień indeksów [45, 5]. W przypadku mowy poniżej 1 kbps wąskie gardło musi być bardzo silne, dlatego projekt koderów i kwantyzatorów staje się ważniejszy niż sama zdolność dekodera do generowania naturalnego brzmienia.

2.6.3 Konwolucje jednowymiarowe w analizie sygnału audio

Konwolucje są jednym z podstawowych mechanizmów przetwarzania sygnałów przez sieci neuronowe. W ujęciu ogólnym warstwa konwolucyjna przesuwając niewielkie jądro po sygnale i stosując ten sam zestaw wag w wielu położeniach. W sieciach przetwarzających obrazy konwolucje zostały spopularyzowane m.in. w pracach LeCuna i współautorów [21], natomiast w przypadku audio analogiczny mechanizm stosuje się wzdłuż osi czasu. Konwolucja jednowymiarowa może wykrywać lokalne wzorce w przebiegu, takie jak narastanie energii, struktury okresowe, fragmenty bezdźwięczne lub szybkie zmiany fazy. Współdzielenie wag jest korzystne, ponieważ te same lokalne zjawiska mogą występować w różnych momentach wypowiedzi.

W kodekach neuronowych konwolucje pełnią zwykle dwie funkcje. Po pierwsze, pozwalają budować lokalny opis sygnału, który jest odporniejszy na drobne przesunięcia w czasie niż analiza pojedynczych próbek. Po drugie, warstwy konwolucyjne z krokiem większym niż jeden umożliwiają redukcję długości sekwencji, a więc zmniejszenie liczby ramek, które trzeba później zakodować. W dekoderyze analogiczną rolę pełni nadpróbkowanie, realizowane na przykład przez transponowane konwolucje lub przez powtarzanie próbek połączone z wygładzającą konwolucją. Przy projektowaniu kodeka strumieniowego istotne są również konwolucje przyczynowe, ponieważ ograniczają wykorzystanie informacji z przyszłości i zmniejszają opóźnienie.

W głębszych modelach konwolucyjnych często stosuje się połączenia rezydualne, które ułatwiają przepływ gradientu i pozwalają budować głębsze bloki bez tak silnej degradacji uczenia [13]. W audio podobny mechanizm może być używany w torze latentnym albo w dekoderyze jako korekta rekonstrukcji. Należy jednak zachować ostrożność: większa liczba warstw poprawia pojemność modelu, ale jednocześnie zwiększa koszt inferencji, pamięć potrzebną na parametry oraz ryzyko przekroczenia ograniczeń urządzenia brzegowego.

2.6.4 Reprezentacja latentna i redukcja czasowa

Reprezentacja latentna jest pośrednim opisem sygnału tworzonym przez koder. W przypadku kompresji audio ma ona zwykle postać sekwencji wektorów, a nie pojedynczego wektora dla całego nagrania. Jeżeli sygnał wejściowy ma długość T próbek, a koder zmniejsza rozdzielczość czasową

S -krotnie, to liczba ramek latentnych wynosi w przybliżeniu T/S . Dla sygnału próbkowanego z częstotliwością F_s częstotliwość ramek latentnych można zapisać jako:

$$f_{\text{lat}} = \frac{F_s}{S}. \quad (2.2)$$

Ta zależność ma bezpośrednie znaczenie dla przepływności bitowej. Jeżeli koder wytwarza mniej ramek na sekundę, to kwantyzator musi przesłać mniej indeksów. Z drugiej strony zbyt silna redukcja czasowa może usunąć informacje potrzebne do wiernej rekonstrukcji spółgłosek, przejść między fonemami i szybkich zmian energii. W praktyce projektowanie reprezentacji latentnej wymaga więc kompromisu między pojemnością informacyjną, opóźnieniem i kosztem dekodowania.

Wymiar wektora latentnego wpływa na to, ile informacji może zostać przechowane w pojedynczej ramce przed kwantyzacją. Nie jest on jednak równoważny przepływności bitowej, ponieważ w kodeku dyskretnym transmitowany jest indeks kodu, a nie pełny wektor zmiennoprzecinkowy. Można więc zwiększać wymiar latentny, aby ułatwić pracę koderowi i dekodekowi, zachowując tę samą nominalną liczbę bitów, o ile liczba słowników i liczba kodów w słownikach pozostają bez zmian. Takie rozróżnienie jest ważne w neuronowych kodekach end-to-end, ponieważ jakość rekonstrukcji zależy zarówno od geometrii przestrzeni latentnej, jak i od dyskretności pojemności kwantyzatora [45, 5].

2.6.5 Kwantyzacja wektorowa i rezydualna kwantyzacja wektorowa

Kwantyzacja wektorowa polega na zastąpieniu ciągłego wektora jednym z elementów skończonego słownika kodowego. Dla słownika $\mathcal{C} = \{e_1, e_2, \dots, e_K\}$ wektor latentny jest przypisywany do najbliższego kodu, a do strumienia bitów trafia tylko indeks wybranego elementu. Klasyczna teoria kwantyzacji wektorowej była rozwijana znacznie wcześniej niż współczesne kodeki neuronowe [11], ale w modelach głębokich problem ten pojawia się ponownie jako sposób uczenia dyskretnych reprezentacji. VQ-VAE pokazał, że kwantyzację można włączyć do autoenkodera i trenować reprezentacje dyskretnie z użyciem estymatora prostego przejścia gradientu oraz dodatkowych składników stabilizujących słownik [41].

Jeżeli pojedynczy słownik ma K kodów, to jeden indeks wymaga nominalnie $\log_2 K$ bitów. Dla wielu słowników, stosowanych w każdej ramce latentnej, przepływność indeksów można zapisać jako:

$$R = f_{\text{lat}} \sum_{i=1}^{N_q} \log_2 K_i, \quad (2.3)$$

gdzie N_q oznacza liczbę etapów kwantyzacji, a K_i liczbę kodów w i -tym słowniku. Widać z tego, że przepływność zależy bezpośrednio od częstotliwości ramek latentnych, liczby słowników oraz ich rozmiaru. Przy projektowaniu kodeka poniżej 1 kbps nawet niewielka zmiana tych parametrów może mieć duże znaczenie, ponieważ budżet bitowy jest bardzo mały.

Rezydualna kwantyzacja wektorowa rozwija ten mechanizm przez użycie wielu etapów. Pierwszy słownik koduje przybliżenie wektora, kolejny koduje błąd pozostały po pierwszym etapie, następny koduje kolejną resztę i tak dalej. Dzięki temu można zwiększyć dokładność rekonstrukcji latentów bez gwałtownego zwiększania rozmiaru pojedynczego słownika. Taki mechanizm jest wykorzystywany w kodekach SoundStream i EnCodec, a warianty grupowej rezydualnej kwantyzacji pojawiają się również w HiFi-Codec [45, 5, 44]. Ograniczeniem RVQ jest możliwość nierównego wykorzystania kodów, czyli sytuacja, w której model wybiera tylko niewielką część słownika. Z tego powodu podczas treningu stosuje się dodatkowe straty, regularyzację wykorzystania kodów albo procedury przeciwdziałające zapadaniu się słowników.

2.6.6 Funkcje aktywacji, normalizacja i regularyzacja

Funkcje aktywacji wprowadzają nieliniowość między kolejnymi warstwami sieci. Bez nich złożenie wielu warstw liniowych nadal byłoby przekształceniem liniowym, a model nie mógłby opisywać złożonych zależności obecnych w sygnale mowy. W praktyce stosuje się różne funkcje aktywacji, takie jak ReLU, GELU, sigmoid, tangens hiperboliczny lub aktywacje dostosowane do struktur okresowych. Wybór aktywacji wpływa na stabilność gradientów, zakres wartości pośrednich oraz zdolność modelu do odtwarzania drobnych szczegółów sygnału [10]. W kodeku audio nie jest to wyłącznie decyzja numeryczna: zbyt agresywna nieliniowość może prowadzić do zniekształceń, a zbyt łagodna może ograniczyć zdolność modelu do rekonstrukcji szybkich zmian w przebiegu.

Normalizacja służy stabilizacji uczenia przez kontrolowanie rozkładu aktywacji. W przetwarzaniu sekwencji często stosuje się normalizację warstwową, która oblicza statystyki dla pojedynczego przykładu, a nie dla całego mini-batcha [3]. Może to być korzystne w modelach audio, ponieważ długość sygnałów, dynamika głośności i zróżnicowanie mówców powodują duże zmiany skali cech. Jednocześnie normalizacja umieszczona bezpośrednio przed kwantyzacją może zmieniać geometrię przestrzeni latentnej, dlatego jej wpływ na stabilność RVQ wymaga weryfikacji eksperymentalnej.

Regularyzacja obejmuje techniki ograniczające przeuczenie i niestabilność modelu. Klasycznym przykładem jest dropout, który losowo wyłącza część aktywacji podczas treningu [37]. W kodekach audio stosowanie takiej regularyzacji wymaga jednak ostrożności, ponieważ celem nie jest klasyfikacja, lecz precyzyjna rekonstrukcja sygnału. Naturalną formą regularyzacji jest również samo wąskie gardło kompresyjne: ograniczona liczba kodów, ograniczona liczba ramek latentnych oraz kary związane z kwantyzacją zmuszają model do wyboru informacji najważniejszych dla rekonstrukcji. W tym sensie architektura kodeka pełni funkcję regularyzującą równie istotną jak klasyczne techniki uczenia głębokiego.

2.6.7 Funkcje strat stosowane w rekonstrukcji audio

Funkcja straty określa, jakie błędy model ma uznawać za najważniejsze podczas treningu. W najprostszym przypadku można minimalizować różnicę między przebiegiem oryginalnym i zrekonstruowanym bezpośrednio w dziedzinie czasu, na przykład za pomocą normy L_1 lub L_2 . Takie podejście dobrze wiąże fazę i amplitudę próbek, ale bywa niewystarczające percepcyjnie. Dwa sygnały o podobnym brzmieniu mogą różnić się punktowo w czasie, a z kolei niski błąd próbkowy nie gwarantuje zachowania struktury harmonicznego i barwy.

Z tego powodu w modelach generujących lub rekonstruujących mowę często stosuje się straty widmowe. Porównują one sygnały po transformacji STFT, dzięki czemu model otrzymuje informację o błędach w dziedzinie czasu i częstotliwości. Szczególnie użyteczne są straty wielorozdzielcze, w których porównanie wykonywane jest dla kilku długości okna i kilku kroków analizy. Takie podejście zostało wykorzystane m.in. w Parallel WaveGAN, gdzie wielorozdzielcza strata spektrogramowa wspierała generowanie fali dźwiękowej [43]. W kodekach neuronowych podobną rolę pełnią straty rekonstrukcyjne, widmowe, percepcyjne i adversarialne, łączone w celu poprawy naturalności odtwarzanego sygnału [45, 5].

W systemie z kwantyzacją konieczne są również składniki związane z samym słownikiem kodowym. Strata przywiązania do kodu, określana w literaturze jako *commitment loss*, ogranicza odrywanie się reprezentacji latentnej od wybranych kodów, natomiast składniki promujące wykorzystanie słownika przeciwdziałają sytuacji, w której większość kodów pozostaje nieużywana. Dodatkowe straty, takie jak podobieństwo kosinusowe przebiegów, straty melowe lub kary za nadmiarową energię w wybranych pasmach, mogą poprawiać wybrane aspekty rekonstrukcji, ale każda z nich wprowadza własny kompromis. Zbyt duża waga strat widmowych może prowadzić do sztucz-

nego wygładzenia lub wzrostu szumu tła, natomiast zbyt silne wymuszanie zgodności w czasie może pogarszać naturalność brzmienia. Dlatego dobór funkcji strat w neuronowym kodeku mowy jest częścią projektu systemu, a nie jedynie technicznym detalem treningu.

2.7 Metody kodowania dźwięku przy bardzo niskich przepływnościach

2.8 Metryki oceny jakości rekonstrukcji dźwięku

Rozdział 3

Metoda i implementacja systemu

Rozdział przedstawia projekt i implementację neuronowego kodeka mowy opracowanego w ramach pracy. Punktem wyjścia są ograniczenia omówione w rozdziale 2: bardzo mała przepływność bitowa, konieczność zachowania zrozumiałości mowy oraz ograniczony koszt obliczeniowy po stronie inferencji. Proponowany system pracuje poniżej 1 kbps, a informacja przesyłana między koderem i dekoderm ma postać indeksów rezydualnej kwantyzacji wektorowej [11, 45, 5]. Nie jest to więc kodek nastawiony na wierne odtworzenie przebiegu próbkowego. Jego zadaniem jest zachowanie tych cech sygnału, które mają największe znaczenie dla odbioru mowy. Do szczegółowego opisu przyjęto wariant wybrany po serii eksperymentów treningowych i odsłuchowych. Odwołuje się on do konkretnego checkpointu oraz zestawu parametrów modelu, dlatego w dalszej części rozdziału jest traktowany jako punkt odniesienia, a nie jako osobny standard kodowania. Model ma około 7,98 mln parametrów, pracuje na sygnale próbkowanym z częstotliwością 16 kHz, redukuje rozdzielczość czasową 256-krotnie i wykorzystuje trzy słowniki RVQ po 32 kody. Z tak dobranych parametrów wynika nominalna przepływność 0,938 kbps dla samych indeksów, bez narzutu nagłówków, kontenera i ewentualnej synchronizacji ramek.

Kolejność opisu odpowiada kolejnym decyzjom projektowym. Najpierw omówiono założenia dotyczące przepływności i zakresu informacji, która powinna przejść przez wąskie gardło kodeka. Następnie opisano reprezentację sygnału wejściowego, architekturę kodera i dekodera oraz sposób zapisu indeksów RVQ. Ostatnia część rozdziału dotyczy implementacji, w tym podziału kodu na backendy, skryptów inferencyjnych, demonstratora przeglądarkowego i testów automatycznych. W tekście rozróżniono tor wymagany w czasie inferencji od mechanizmów używanych tylko podczas treningu lub diagnostyki.

3.1 Założenia projektowe proponowanego rozwiązania

System zaprojektowano jako neuronowy kodek mowy dla przepływności poniżej 1 kbps. Przy tak małej liczbie bitów na sekundę nie można traktować zgodności próbek jako podstawowego kryterium jakości. Kanał jest zbyt wąski, aby przenieść pełny opis przebiegu czasowego lub bogaty opis widmowy. Założono więc, że model powinien zachowywać przede wszystkim informację fonetyczną, rytm wypowiedzi, podstawową barwę głosu i zarys struktury harmoniczej. Szczegóły akustyczne o mniejszym znaczeniu percepcyjnym mogą być rekonstruowane jedynie w przybliżeniu. Model referencyjny przetwarza mowę próbkowaną z częstotliwością 16 kHz. Koder zmniejsza rozdzielczość czasową 256-krotnie, dlatego jedna sekunda sygnału odpowiada 62,5 ramkom latentnym. Każda ramka jest opisywana przez trzy indeksy RVQ, po jednym z każdego słownika o rozmiarze 32. Pojedynczy indeks wymaga nominalnie 5 bitów, a cała ramka 15 bitów. Przepływność strumie-

nia indeksów wynosi więc $62,5 \cdot 15 = 937,5$ bit/s, czyli 0,938 kb/s. Jest to wartość odnosząca się wyłącznie do właściwego ładunku RVQ.

Parametr	Wartość w przyjętym wariantcie modelu
Częstotliwość próbkowania	16 kHz
Długość segmentu treningowego	16000 próbek, czyli jedna sekunda audio
Redukcja czasowa kodera	$256\times$ względem fali wejściowej
Częstość ramek latentnych	62,5 ramki na sekundę
Kwantyzacja	3 słowniki RVQ po 32 kody
Nominalna przepływność indeksów	937,5 bit/s
Liczba parametrów modelu	około 7,98 mln

TABELA 3.1: Parametry przyjętego wariantu modelu.

Źródło: opracowanie własne na podstawie pliku konfiguracyjnego i checkpointu modelu; liczba parametrów została wyznaczona przez zsumowanie parametrów warstw, a przepływność dotyczy nominalnego ładunku indeksów RVQ.

Ograniczenie przepływności wpływa na cały układ modelu. Koder musi sprowadzić falę mowy do krótkiej sekwencji wektorów latentnych, a kwantyzator zamienia te wektory na indeksy ze skończonych słowników. Dekoder nie ma dostępu do oryginalnego sygnału ani do ciągłej reprezentacji przed kwantyzacją. Odtwarza przebieg czasowy wyłącznie ze skwantowanych latentów. Opisany model łączy autoenkoderowy podział na koder i dekodek, rezydualną kwantyzację wektorową oraz jednowymiarowe warstwy konwolucyjne [45, 5]. Do toru inferencji należą tylko koder, kwantyzator i dekodek. Wieloskalowe straty STFT, straty melowe, składniki semantyczne oparte na HuBERT lub ContentVec [15, 32] oraz metryki wykorzystania słowników służą do uczenia i kontroli modelu, ale nie są przesyłane w strumieniu bitowym. Osobnym ograniczeniem jest stabilność kwantyzacji. Przy trzech słownikach po 32 kody zapadnięcie się jednego słownika zmniejsza rzeczywistą pojemność kanału, mimo że nominalna przepływność pozostaje taka sama [41, 45, 5]. Z tego powodu w eksperymentach śledzono wykorzystanie kodów, entropię słowników i składniki straty związane z RVQ. Projekt wymagał kompromisu między jakością rekonstrukcji, zgodnością widmową, stabilnością słowników i kosztem obliczeniowym. Większy model może lepiej rekonstruować mowę, ale gorzej odpowiada wymaganiom urządzeń brzegowych. Mniejsza liczba kodów ogranicza bitrate, lecz zwiększa ryzyko utraty zrozumiałości.

3.2 Dobór reprezentacji sygnału wejściowego

Koder otrzymuje przebieg czasowy mowy. Reprezentacje widmowe pojawiają się dopiero w funkcji straty, gdzie służą do oceny błędów rekonstrukcji. Taki podział upraszcza strumień bitowy, a jednocześnie umożliwia karanie błędów słyszalnych w dziedzinie częstotliwości.

3.2.1 Fala mowy jako wejście modelu

Model otrzymuje na wejściu bezpośrednio przebieg czasowy mowy. Nie wprowadzono ręcznie wyznaczanych parametrów aparatu mowy, współczynników LPC ani spektrogramu jako wejścia kodera. Dla pojedynczej próbki treningowej sygnał można zapisać jako sekwencję

$$x = (x_1, x_2, \dots, x_T),$$

gdzie T oznacza liczbę próbek w segmencie. W implementacji tensor audio ma układ $(B, T, 1)$, czyli obejmuje mini-batch, oś czasu i jeden kanał sygnału. Ten sam format jest używany na wejściu kodera i na wyjściu dekodera.

3.2.2 Próbkowanie, segmentacja i normalizacja

Opisywany model pracuje na mowie próbkowanej z częstotliwością 16 kHz. Pasma Nyquista wynosi wtedy 8 kHz, co odpowiada szerokopasmowej mowie i pozwala oceniać zarówno zrozumiałość, jak i podstawową naturalność rekonstrukcji. Wyższa częstotliwość próbkowania zwiększyłaby liczbę próbek przetwarzanych w każdej sekundzie, a tym samym koszt kodera i dekodera. Niższa częstotliwość ograniczyłaby pasmo jeszcze przed kodowaniem, przez co porównanie jakości byłoby mniej miarodajne. W głównych uruchomieniach treningowych stosowano segmenty o długości 16000 próbek, czyli jedną sekundę audio. Po redukcji czasowej $256\times$ taki segment odpowiada około 62,5 ramkom latentnym.

Pliki audio są ładowane rekurencyjnie z katalogu danych i sprowadzane do jednego kanału. Jeżeli nagranie ma więcej kanałów, wykorzystywany jest pierwszy z nich. Próbkki krótsze od wymaganego segmentu są dopełniane zerami, a dłuższe są przycinane. Gdy częstotliwość próbkowania pliku różni się od wartości docelowej, sygnał jest dopasowywany do długości segmentu dla 16 kHz. Po wczytaniu każdy segment jest normalizowany względem maksymalnej wartości bezwzględnej amplitudy:

$$x \leftarrow \frac{x}{\max_t |x_t| + 10^{-5}}.$$

Normalizacja ogranicza wpływ różnic głośności między nagraniami i stabilizuje zakres wartości wejściowych. Nie usuwa lokalnej dynamiki wypowiedzi, ponieważ współczynnik skalowania jest wspólny dla całego segmentu.

Reprezentacja	Miejsce użycia	Rola w systemie
Fala audio $(B, T, 1)$	Wejście i wyjście kodeka	Podstawowy format inferencji i rekonstrukcji
Latenty kodera	Przed RVQ	Zwarty opis sygnału przed dyskretyzacją
Indeksy RVQ	Strumień kompresji	Informacja przesyłana między koderem i dekodere
STFT	Funkcja straty	Kontrola struktury widmowej w kilku skalach
Skala mel	Funkcja straty	Pomocnicza ocena błędów zgodna z percepcją pasmową

TABELA 3.2: Reprezentacje sygnału wykorzystywane w treningu i inferencji.

Źródło: opracowanie własne na podstawie toru treningu i inferencji przyjętego w implementacji.

3.2.3 Reprezentacje pomocnicze w funkcji straty

Praca bezpośrednio na fali wymaga odpowiedniego doboru funkcji kosztu. Sama strata L_1 w dziedzinie czasu wiąże fazę i amplitudę próbek, ale nie opisuje wprost jakości widmowej. Niewielkie przesunięcie fazowe może zwiększyć błąd próbkowy mimo podobnego brzmienia, a mały błąd w czasie nie gwarantuje zachowania struktury harmonicznej. W treningu zastosowano więc pomocnicze reprezentacje STFT i mel, liczone osobno dla sygnału oryginalnego i zrekonstruowanego [43, 45, 5]. Model nie koduje ramek STFT ani mel-spektrogramu. Są one używane tylko do wyznaczania składników straty. W głównych ustawieniach eksperymentalnych stosowano wielorozdzielczą analizę STFT z parami (n_{fft}, h) równymi (512, 128), (1024, 256), (2048, 512) oraz (4096, 1024), gdzie h oznacza krok analizy. Krótsze okna opisują lokalne zmiany w czasie, natomiast dłuższe

lepiej ujmują strukturę harmoniczną i rozkład energii w częstotliwości. Użyto także logarytmicznej reprezentacji melowej o 80 pasmach, liczonej dla $n_{\text{fft}} = 1024$ i kroku 256. Wejście i wyjście kodeka pozostają zatem jednowymiarową falą mowy, a dodatkowe widoki sygnału występują tylko w procesie uczenia.

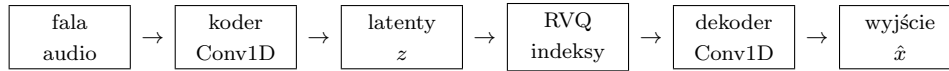
Widok	Parametry	Waga	Znaczenie w treningu
STFT	$n_{\text{fft}} = 512, h = 128$	0,8	Krótkie zmiany lokalne i początek głosek
STFT	$n_{\text{fft}} = 1024, h = 256$	1,1	Podstawowy kompromis czasu i częstotliwości
STFT	$n_{\text{fft}} = 2048, h = 512$	1,8	Stabilizacja struktury harmonicznego
STFT	$n_{\text{fft}} = 4096, h = 1024$	3,0	Silniejsza kontrola dłuższych składowych widma
Mel	80 pasm, 1024/256	$\lambda = 0,055$	Percepcyjny rozkład błędów w paśmie 0-8 kHz

TABELA 3.3: Widmowe reprezentacje pomocnicze używane w treningu.

Źródło: opracowanie własne na podstawie ustawień funkcji straty. Parametry STFT i mel nie są elementami strumienia bitowego; służą wyłącznie do obliczania strat podczas treningu.

3.3 Architektura koder a i dekodera

Architektura ma postać autoenkodera z dyskretnym wąskim gardłem. Koder zamienia falę mowy na sekwencję latentów, kwantyzator ogranicza tę sekwencję do indeksów RVQ, a dekoderek rekonstruuje przebieg czasowy. Taki układ odpowiada ogólnej konstrukcji neuronowych kodeków audio end-to-end [45, 5]. Ogólny tor przetwarzania pokazano na Rysunku 3.1.



RYСУNEK 3.1: Ogólny tor kodowania i dekodowania w opisywanym modelu.

3.3.1 Tor koder a i reprezentacja latentna

Koder przekształca sygnał wejściowy x w sekwencję latentów z . Następnie rezydualny kwantyzator wektorowy przypisuje te wektory do skończonych słowników, a dekoderek odtwarza przebieg $\hat{x}$. Zależność między modułami można zapisać następująco:

$$z = E_{\theta}(x), \quad z_q = Q(z), \quad \hat{x} = D_{\phi}(z_q),$$

gdzie E_{θ} oznacza koder, Q moduł kwantyzacji, a D_{ϕ} dekoderek. Po stronie dekodera nie jest dostępny pełny wektor ciągły z . Rekonstrukcja powstaje wyłącznie na podstawie z_q , czyli reprezentacji zbudowanej z wybranych kodów RVQ.

Koder składa się z jednowymiarowych warstw konwolucyjnych pracujących wzdłuż osi czasu. Każdy główny etap zmniejsza długość sekwencji dwukrotnie, dlatego osiem etapów daje całkowitą redukcję $2^8 = 256$. W przyjętym wariancie szerokości kolejnych warstw wynoszą:

$$(24, 32, 48, 64, 96, 128, 192, 256).$$

Po ostatnim etapie konwolucyjnym wynik jest rzutowany do przestrzeni latentnej o wymiarze 512. Dla sygnału próbkowanego z częstotliwością 16 kHz taka redukcja daje nominalnie 62,5 ramek latentnych na sekundę. W warstwach koder a zastosowano aktywację `snake.beta`, która łączy składnik liniowy ze składnikiem sinusoidalnym [48, 20]. Jest to użyteczne w modelu audio, ponieważ sygnał mowy zawiera struktury okresowe związane z drganiami fałdów głosowych i harmonicznymi.

Etap	Liczba kanałów	Redukcja po etapie	Czas jednej pozycji przy 16 kHz
1	24	$2\times$	0,125 ms
2	32	$4\times$	0,25 ms
3	48	$8\times$	0,5 ms
4	64	$16\times$	1 ms
5	96	$32\times$	2 ms
6	128	$64\times$	4 ms
7	192	$128\times$	8 ms
8	256	$256\times$	16 ms

TABELA 3.4: Redukcja czasowa w kolejnych etapach kodera opisywanego modelu.

Źródło: opracowanie własne na podstawie architektury kodera; czasy wynikają z próbkowania 16 kHz.

3.3.2 Kontekst latentny i kwantyzacja RVQ

Za koderem umieszczono krótki tor przetwarzania latentów. Przyjęty wariant wykorzystuje dwa bloki temporalne przed RVQ oraz dwa bloki po RVQ. Są to rezydualne konwolucje dylatowane działające na sekwencji latentnej. Zwiększają kontekst czasowy bez zmiany liczby ramek i bez zwiększenia nominalnej przepływności [13]. Przed kwantyzacją zastosowano także pojedynczy blok samouwagi z dwoma głowami, aby powiązać bardziej odległe fragmenty krótkiego segmentu latentnego [42]. Kwantyzator składa się z trzech etapów rezydualnej kwantyzacji wektorowej. Pierwszy etap wybiera kod przybliżający latent, drugi koduje resztę po pierwszym przybliżeniu, a trzeci resztę pozostałą po dwóch wcześniejszych etapach. Każdy etap ma słownik o 32 kodach. Mechanizm ten rozwija klasyczną kwantyzację wektorową i jest typowy dla współczesnych kodeków neuronowych z dyskretnym wąskim gardłem [11, 41, 45]. W implementacji użyto odległości cosinusowej oraz projekcji kodów do wymiaru 8. Dobór indeksu odbywa się więc w przestrzeni pomocniczej o niższym wymiarze, a wynik kwantyzacji jest rzutowany z powrotem do wymiaru latentnego dekodera. Takie rozwiązanie zmniejsza koszt słowników bez zmiany liczby transmitowanych indeksów.

Element	Wartość	Konsekwencja dla kodeka
Liczba etapów RVQ	3	Każda ramka ma trzy indeksy dyskretne
Rozmiar słownika	32 kody	Jeden indeks wymaga 5 bitów
Bitów na ramkę latentną	15 bitów	$3 \cdot \log_2 32$ dla każdej ramki
Wymiar latentny	512	Wymiar reprezentacji przekazywanej do toru RVQ
Wymiar kodu RVQ	8	Wybór kodu odbywa się w mniejszej przestrzeni pomocniczej
Metryka wyboru kodu	cosinusowa	Dobór kodu zależy od kierunku wektora latentnego

TABELA 3.5: Parametry kwantyzacji RVQ w opisywanym modelu.

Źródło: opracowanie własne na podstawie parametrów kwantyzatora RVQ w opisywanym modelu.

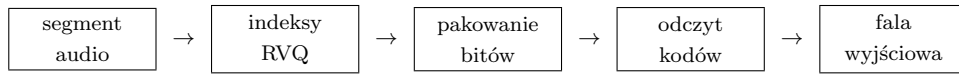
3.3.3 Dekoder konwolucyjny i post-filtr

Dekoder wykonuje operację odwrotną do redukcji czasowej kodera. Na wejściu otrzymuje sekwencję skwantowanych latentów i przeprowadza osiem etapów zwiększania rozdzielczości czasowej. W tym wariancie zastosowano tryb `repeat_conv`: próbki w osi czasu są najpierw powielane, a następnie wygładzane konwolucją jednowymiarową. Rozwiązanie to jest prostsze od transponowanej konwolucji i ogranicza ryzyko regularnych artefaktów związanych z nierównomiernym pokryciem próbek. Liczba kanałów dekodera zmienia się w odwrotnej kolejności względem kodera, od 256 do 24, a ostatnia konwolucja sprowadza wynik do jednego kanału audio. Po głównym torze dekodera zastosowano niewielki rezydualny blok korekcyjny oraz post-filtr działający już na fali wyjściowej. Blok dekodera ma głębokość 1 i wagę rezydualną 0,018, dlatego pełni rolę lokalnej korekty, a nie osobnego generatora. Post-filtr składa się z dwóch bloków konwolucyjnych, ma 24 kanały, jądro długości 15 oraz wagę 0,02. Jego reszta jest filtrowana wysokoprzepustowo przed dodaniem do rekonstrukcji. Implementacja zawiera również mechanizmy opcjonalne, między innymi głośnie pasmowe dekodera, źródło harmoniczne, tryb przyczynowy i pomocnicze głośnie sterujące resztą mowy. Nie są one jednak częścią podstawowego toru opisywanego wariantu.

Znaczenie toru dekodera jest inne niż w klasycznym autoenkoderze bez kompresji. Dekoder nie może polegać na tym, że latent zawiera szczegółowy opis fali, ponieważ po stronie odbiornika pozostają wyłącznie trzy indeksy dla każdej ramki. Pojedyncza ramka odpowiada około 16 ms mowy, a więc obejmuje fragment, w którym mogą występować krótkie przejścia artykulacyjne. Z tego powodu kolejne etapy zwiększania rozdzielczości muszą jednocześnie odtwarzać gładki przebieg czasowy i nie zacierać szybkich zmian energii. W praktyce oznacza to, że dekodek jest odpowiedzialny za dużą część lokalnego modelowania akustycznego, natomiast kwantyzator decyduje o tym, jaka informacja może w ogóle zostać przekazana do tego modelowania. Wariant `repeat_conv` wybrano również ze względu na kontrolę artefaktów. Transponowana konwolucja może wprowadzać regularne wzory w sygnale wyjściowym, jeżeli jądro i krok nie pokrywają osi czasu równomiernie. Przy mowie takie artefakty bywają słyszane jako chropowatość, metaliczność albo okresowe szумы w tle. Powielanie próbek przed konwolucją nie usuwa wszystkich problemów rekonstrukcji, ale oddziela samo zwiększenie długości sekwencji od późniejszego wygładzania. Dzięki temu łatwiej analizować, czy błąd pochodzi z kwantyzacji, z upsamplingu, czy z ostatnich warstw generujących falę. Niewielka waga post-filtru była celową decyzją. Przy większym wzmocnieniu filtr mógłby poprawiać pozorną ostrość sygnału, ale jednocześnie zwiększać energię szumu i maskować błędy wynikające z wcześniejszych etapów. W opisywanym wariancie post-filtr ma korygować drobną resztę wysokoczęstotliwościową po rekonstrukcji, a nie zastępować dekodek. Taki układ jest też korzystny dla diagnostyki: podstawową jakość modelu można oceniać na wyjściu głównego dekodera, a wpływ post-filtru pozostaje ograniczony i łatwy do wyłączenia w eksperymencie kontrolnym.

3.4 Proces kompresji i rekonstrukcji sygnału

W trybie plikowym kompresja polega na zamianie segmentów audio na ciąg indeksów RVQ. Rekonstrukcja wykonuje operację odwrotną: odczytuje indeksy, odtwarza skwantowane latenty i przekazuje je do dekodera. Oba etapy muszą zachować zgodność długości sygnału, parametrów segmentacji oraz formatu pakowania indeksów. Schemat procesu pokazano na Rysunku 3.2. Opis procesu rozdzielono od samej architektury, ponieważ w praktycznym kodeku równie ważne jak model są operacje wykonywane przed i po nim. Segmentacja decyduje o tym, jak długie fragmenty trafiają do kodera. Pakowanie indeksów decyduje o rzeczywistej objętości zapisu. Odtworzenie długości sygnału decyduje natomiast o tym, czy rekonstrukcja może być porównana z nagraniem wejściowym bez dodatkowych przesunięć lub dopełnień. Te elementy nie zwiększają liczby parametrów modelu, ale mogą zmienić wynik pomiaru przepływności i jakość odsłuchową krótkich nagrań.



RYSUNEK 3.2: Kolejne etapy kompresji i rekonstrukcji sygnału w trybie plikowym.

3.4.1 Przygotowanie i kodowanie segmentu

Kompresja rozpoczyna się od przygotowania fragmentu sygnału audio. W implementacji ma on postać tensora $(1, T, 1)$. W trybie plikowym dłuższe nagranie jest dzielone na segmenty o długości zgodnej z ustawieniami treningu. W opisywanym wariancie jest to 16000 próbek. Jeżeli ostatni segment jest krótszy, zostaje dopełniony zerami tylko na czas przejścia przez model. Po rekonstrukcji zachowywana jest część odpowiadająca rzeczywistej długości wejścia, aby zapisany wynik nie zawierał sztucznego ogona wynikającego z dopełnienia.

Właściwe kodowanie zaczyna się od przejścia przez koder:

$$z = E_{\theta}(x).$$

Sekwencja z ma mniejszą rozdzielczość czasową niż fala wejściowa. Dla redukcji $256\times$ jedna ramka latentna odpowiada w przybliżeniu 256 próbkom sygnału, czyli 16 ms przy próbkowaniu 16 kHz. Następnie każda ramka latentna trafia do RVQ. Dla ramki t kwantyzator zwraca zestaw indeksów:

$$i_t = (i_{t,1}, i_{t,2}, i_{t,3}),$$

gdzie każdy indeks wskazuje jeden kod z odpowiedniego słownika. Pierwszy słownik opisuje główne przybliżenie wektora latentnego, a kolejne słowniki opisują resztę po poprzednich etapach. W strumieniu kompresji zapisywane są indeksy $i_{t,j}$, nie zaś same wektory kodowe.

Długość segmentu 16000 próbek przyjęto jako kompromis między kosztem treningu i długością kontekstu. Jednosekundowy fragment zawiera zwykle kilka głosek lub krótką frazę, więc model widzi przejścia między elementami wypowiedzi, a nie tylko izolowane wycinki sygnału. Jednocześnie taki segment jest na tyle krótki, że pozwala utrzymać rozsądny rozmiar mini-batcha i często zapisywać próbki odsłuchowe. W trybie plikowym granice segmentów nie są osobno kodowane jako informacja semantyczna. Są tylko technicznym podziałem nagrania na porcje możliwe do przetworzenia przez model. Dopełnianie ostatniego segmentu zerami wymaga ostrożnego obcięcia rekonstrukcji. Jeżeli dopełnienie pozostałoby w pliku wynikowym, obiektywne metryki porównujące sygnał wejściowy i wyjściowy liczyłyby również fragment, którego nie było w oryginale. Z tego powodu liczba rzeczywistych próbek jest przechowywana w metadanych strumienia i wykorzystywana

przy zapisie fali wyjściowej. W demonstratorze przeglądarkowym podobny problem występuje przy porcjach mikrofonowych: bufor może gromadzić próbki krócej niż jedna sekunda, ale model nadal musi otrzymać poprawnie uformowany tensor i zwrócić rekonstrukcję o kontrolowanej długości.

3.4.2 Przepływność nominalna i pakowanie indeksów

Nominalna przepływność zależy od liczby ramek latentnych na sekundę oraz liczby bitów potrzebnych do zapisu indeksów. Dla ogólnego przypadku można ją zapisać jako:

$$R = \frac{f_s}{S} \sum_{j=1}^{N_q} \log_2 K_j,$$

gdzie f_s jest częstotliwością próbkowania, S całkowitym współczynnikiem redukcji czasowej, N_q liczbą słowników RVQ, a K_j rozmiarem j -tego słownika. Dla przyjętych parametrów wartości te wynoszą $f_s = 16000$, $S = 256$, $N_q = 3$ oraz $K_j = 32$. Otrzymuje się zatem:

$$R = \frac{16000}{256} (5 + 5 + 5) = 937,5 \text{ bit/s.}$$

Jest to wartość nominalna dla samych indeksów. Nie obejmuje nagłówka pliku, synchronizacji ramek, zabezpieczeń transmisji ani dodatkowych znaczników wymaganych przez protokół zapisu.

W implementacji inferencyjnej indeksy mogą być zapisane do pliku `*_codes.bin`. Nie są wtedy przechowywane jako liczby 16- lub 32-bitowe. Zamiast tego stosowane jest pakowanie z szerokością równą $\lceil \log_2 K_j \rceil$. Dla słownika $K = 32$ oznacza to 5 bitów na indeks i 15 bitów na ramkę latentną. Plik zawiera również nagłówek z metadanymi: liczbą słowników, rozmiarem słownika, liczbą ramek latentnych, częstotliwością próbkowania i liczbą próbek audio. Format ten wykorzystano do porównywania przepływności nominalnej z rozmiarem praktycznego zapisu.

Pole	Rozmiar	Znaczenie
Identyfikator formatu	4 bajty	Rozpoznanie pliku z indeksami kodeka
Wersja	1 bajt	Wariant nagłówka; dla równych słowników używana jest wersja 1
Liczba słowników	2 bajty	Liczba etapów RVQ zapisanych w każdej ramce
Rozmiar słownika	4 bajty	Wspólne K dla wszystkich etapów RVQ
Liczba ramek latentnych	4 bajty	Długość zakodowanej sekwencji indeksów
Częstotliwość próbkowania	4 bajty	Częstotliwość rekonstrukcji audio
Liczba próbek audio	4 bajty	Rzeczywista długość sygnału przed dopełnieniem segmentu
Liczba bitów na indeks	1 bajt	Dla $K = 32$ jest to 5 bitów
Ładunek z indeksami	zmienna	Kolejne indeksy q_0, q_1, q_2 pakowane bit po bicie

TABELA 3.6: Pola zapisu `*_codes.bin` dla równych rozmiarów słowników RVQ.

Źródło: opracowanie własne na podstawie formatu pakowania implementowanego w narzędziu inferencyjnym.

Nagłówek jest mały względem dłuższych nagrań, ale ma znaczenie przy bardzo krótkich próbkach testowych. Dlatego w wynikach inferencji rozróżniano przepływność nominalną i przepływność praktycznego zapisu. Pierwsza wynika bezpośrednio z liczby ramek oraz rozmiaru słowników. Druga obejmuje rzeczywistą liczbę bajtów w pliku `*_codes.bin`. Dla kilkusekundowej próbki różnica między tymi wartościami jest niewielka, natomiast dla pojedynczego krótkiego zdania stały nagłówek może chwilowo zawyżyć pomiar. Z tego powodu nominalny bitrate jest lepszą miarą własności modelu, a rozmiar pliku lepiej opisuje koszt konkretnego zapisu. Zapis indeksów jako liczb

16-bitowych byłby prostszy, ale nie odpowiadałby założeniom pracy. Dla trzech słowników po 32 kody każda ramka potrzebuje 15 bitów. Zapis `uint16` zużywałby 48 bitów na ramkę, czyli podnosiłby sam ładunek indeksów do 3 kbps przy tej samej architekturze. Pakowanie bitowe usuwa ten narzut i pozwala sprawdzić, czy model rzeczywiście mieści się poniżej progu 1 kbps. Jest to szczególnie ważne przy porównywaniu wariantów różniących się rozmiarem słowników, ponieważ samo zmniejszenie liczby kodów nie daje efektu, jeżeli późniejszy format zapisu marnuje stałą szerokość typu całkowitego.

3.4.3 Rekonstrukcja po stronie dekodera

Dekodowanie rozpoczyna się od odczytania indeksów i odtworzenia odpowiadających im wektorów ze słowników RVQ. Dekoder nie otrzymuje oryginalnych latentów z , lecz wyłącznie z_q zbudowane ze skwantowanych wkładów. Następnie działa tor post-RVQ i dekodek konwolucyjny:

$$\hat{x} = D_\phi(z_q).$$

Dekoder zwiększa rozdzielczość czasową przez kolejne etapy `repeat_conv`, sprowadza wynik do jednego kanału audio i ogranicza zakres amplitudy funkcją `tanh`. Jeżeli aktywny jest post-filtr fali, działa dopiero po podstawowej rekonstrukcji. Nie zmienia strumienia indeksów ani nominalnej przepływności, ponieważ po stronie transmisji nie pojawia się dodatkowa informacja. Przy przepływności subkilobitowej rekonstrukcja nie jest odtworzeniem próbkowym wejścia. Jest wynikiem działania modelu, który ma zachować treść wypowiedzi i podstawowe cechy brzmienia mimo bardzo małej liczby bitów przypadających na ramkę latentną.

W praktycznej implementacji dekodek musi także odtworzyć kształt sekwencji latentnej. Liczba ramek zapisana w nagłówku strumienia określa, ile zestawów indeksów należy odczytać z ładunku bitowego. Liczba próbek audio określa natomiast, do jakiej długości przyciąć falę po przejściu przez model. Te dwie wartości nie są zamienne. Liczba ramek wynika z pracy kodera i redukcji czasowej, a liczba próbek opisuje oryginalny sygnał przed dopełnieniem. Rozdzielenie tych pól ułatwia obsługę nagrań, których długość nie jest wielokrotnością kroku kodera. Format strumienia nie zawiera mechanizmów korekcji błędów. Jeżeli bit w ładunku zostanie uszkodzony, może zmienić indeks jednego ze słowników dla danej ramki. W trybie plikowym zakłada się poprawny zapis i odczyt danych, więc taki przypadek jest traktowany jako błąd warstwy transportowej. Dla transmisji sieciowej należałoby dodać osobny mechanizm wykrywania utraty porcji, numerowania ramek lub sum kontrolnych. Nie zmienia to architektury kodeka, ale wpływa na protokół otaczający model. W demonstratorze mikrofonowym tę rolę częściowo pełnią numery sekwencji porcji oraz statystyki kolejki, utraconych pakietów i niedopełnień bufora.

Ograniczenie do samych indeksów RVQ ma też konsekwencje jakościowe. Dekoder może wygenerować sygnał spójny akustycznie, ale nie ma dostępu do informacji, której kwantyzator nie przeniósł. Jeżeli wąskie gardło utraci rozróżnienie między podobnymi fragmentami fonetycznymi, późniejsze warstwy mogą jedynie wybrać najbardziej prawdopodobną rekonstrukcję dla danego kodu. Z tego powodu analiza modelu nie może kończyć się na działaniu dekodera. Trzeba równolegle oceniać wykorzystanie słowników, stabilność wyboru kodów i to, czy kolejne etapy RVQ rzeczywiście kodują resztę po poprzednich etapach. Wariant plikowy nie wymusza ścisłej przyczynowości. Koder i dekodek mogą korzystać z kontekstu dostępnego w przetwarzanym segmencie, a wynik jest zapisywany dopiero po zakończeniu przejścia przez model. Wariant strumieniowy ma ostrzejsze ograniczenia, ponieważ opóźnienie zależy od długości porcji mikrofonowej, czasu resamplingu, inferencji i przesłania odpowiedzi przez WebSocket. Dlatego w demonstratorze mierzono nie tylko nominalny bitrate, ale także czas ścienny, RTF, szybkość względem czasu rzeczywistego

oraz długość kolejki. Te wielkości pokazują, czy model nadaje się do interaktywnego użycia, a nie tylko do przetwarzania gotowego pliku.

3.5 Implementacja systemu oraz wykorzystane technologie

Implementację przygotowano tak, aby jeden zestaw ustawień mógł opisywać trening, inferencję i narzędzia diagnostyczne. Repozytorium zawiera backend CUDA/PyTorch, pomocniczy backend MLX, skrypty inferencyjne, lokalny demonstrator przeglądarkowy oraz testy automatyczne [25, 27, 1]. Główne elementy tej struktury zestawiono w Tabeli 3.7.

Element	Rola w projekcie
<code>config.py</code>	Wspólna konfiguracja architektury, treningu i obliczania przepływności
<code>cuda/</code>	Backend PyTorch/CUDA, model, dane, straty i trening
<code>mlx/</code>	Backend pomocniczy używany do lokalnych eksperymentów i inferencji na Apple Silicon
<code>tools/</code>	Skrypty inferencji, konwersji checkpointów i demonstratora przeglądarkowego
<code>runs/, experiments/</code>	Katalogi wynikowe z checkpointami, logami, spektrogramami i plikami audio
<code>Dockerfile</code>	Opis środowiska uruchomieniowego dla treningu CUDA
<code>tests/</code>	Testy konfiguracji, pakowania indeksów, protokołu demonstratora i kontrolnych przejść modelu

TABELA 3.7: Moduły i artefakty implementacji systemu.

Źródło: opracowanie własne na podstawie struktury repozytorium projektu.

3.5.1 Układ repozytorium i backendy

System zaimplementowano jako pakiet Pythona `sirencodec` w układzie katalogów `src/`. Kod podzielono na część wspólną, backend treningowy CUDA/PyTorch, pomocniczy backend MLX oraz narzędzia uruchomieniowe. Podział ten wynikał ze sposobu prowadzenia eksperymentów. Najbardziej obciążające uruchomienia wykonywano na maszynie z kartą NVIDIA, natomiast krótsze testy odsłuchowe i inferencję lokalną prowadzono na Apple Silicon [1]. Wspólne pozostały ustawienia kodeka, obliczanie przepływności, liczba słowników RVQ i geometria kodera. Główny backend znajduje się w katalogu `cuda/` i korzysta z bibliotek PyTorch oraz TorchAudio [27, 31]. Warstwa danych używa standardowych plików audio, a operacje pomocnicze wykorzystują między innymi `numpy`, `soundfile` i `matplotlib` [12, 30, 16]. PyTorch odpowiada za model, funkcje strat, propagację gradientu i zapis stanu treningu. Główny wariant rozwijano przede wszystkim w backendzie MLX z katalogu `mlx/`. Backend ten odtwarza tę samą strukturę kodeka, lecz jest dostosowany do uruchomień na Apple Silicon. Służył do szybkiej inferencji, odsłuchu kolejnych checkpointów i lokalnych eksperymentów z ustawieniami strat.

3.5.2 Artefakty treningu i inferencja

Uruchomienia treningowe tworzą katalogi wynikowe w `runs/` lub `experiments/`. Zapisywane są w nich checkpointy, logi tekstowe, logi tabelaryczne, pliki `results.tsv`, konfiguracja treningu, stan wznowienia oraz materiały odsłuchowe. Dla wybranych kroków eksportowane są spektrogramy i pary plików audio z sygnałem oryginalnym oraz zrekonstruowanym. Rozdzielenie tych artefaktów było konieczne, ponieważ sam spadek funkcji kosztu nie wystarczał do oceny kodeka mowy. Logi pozwalały śledzić wykorzystanie słowników, straty widmowe i koszt kwantyzacji, a pliki audio służyły do kontroli słyszalnych artefaktów. Inferencję plikową wydzielono do narzędzi pomocniczych. Skrypt `infer_mlx.py` ładuje checkpoint MLX, dzieli nagranie na segmenty zgodne z ustawieniami treningu, wykonuje kodowanie i dekodowanie, a następnie zapisuje rekonstrukcję. Może także zapisywać indeksy RVQ w postaci spakowanego strumienia `*_codes.bin`. Do testowania pracy interaktywnej przygotowano lokalny demonstrator przeglądarkowy. Moduł `web_streaming.py` definiuje format porcji audio przesyłanych z przeglądarki do backendu, a skrypt `web_stream_demo.py` ładuje wybrany checkpoint, odbiera porcje przez WebSocket, wykonuje resampling i zwraca rekonstrukcję wraz z podstawowymi statystykami czasu przetwarzania.

W repozytorium rozróżniono checkpointy pełne i checkpointy przeznaczone do inferencji. Pliki `.safetensors` przechowują stan treningu w formie potrzebnej do wznowienia eksperymentu, natomiast pliki `codec_step*.npz` są wygodnym formatem dla backendu MLX i narzędzi odsłuchowych. Obok checkpointów zapisywane są pliki metadanych, komenda treningowa i konfiguracja uruchomienia. Takie rozdzielenie zmniejsza ryzyko pomylenia modelu gotowego do inferencji ze stanem optymalizatora albo wariantem, który powstał tylko jako pośredni punkt treningu. Materiały odsłuchowe były zapisywane regularnie, zwykle razem ze spektrogramami. Para plików `orig.wav` i `recon.wav` pozwalała szybko sprawdzić, czy poprawa liczby w logu rzeczywiście przekłada się na mowę. W tym projekcie było to potrzebne, ponieważ część strat może poprawiać wybrane metryki i jednocześnie pogarszać odbiór rekonstrukcji. Przykładem jest nadmierne wzmacnianie składowych wysokoczęstotliwościowych, które może zmniejszać błąd widmowy w pewnym zakresie, ale w odsłuchu daje szorstki szum. Z tego powodu checkpoint referencyjny wybierano nie tylko na podstawie pojedynczej wartości funkcji kosztu. Demonstrator przeglądarkowy pełnił rolę testu wdrożeniowego. Przeglądarka zbiera próbki z mikrofonu, wysyła je w porcjach przez WebSocket, a backend odsyła zrekonstruowane audio i statystyki. W logu interfejsu widoczna jest liczba wysłanych porcji, długość kolejki, spadki, niedopełnienia bufora, nominalna przepływność, praktyczna przepływność ładunku oraz RTF. Te dane pozwalają szybko rozdzielić problemy modelu od problemów transportu. Jeżeli rośnie kolejka, ograniczeniem jest czas przetwarzania albo komunikacja. Jeżeli kolejka jest pusta, a pojawiają się niedopełnienia, problem dotyczy dostarczania lub harmonogramu odtwarzania audio po stronie przeglądarki.

3.5.3 Środowisko uruchomieniowe i testy

Środowisko uruchomieniowe opisano w pliku `Dockerfile`. Obraz bazuje na Pythonie 3.12, instaluje `ffmpeg`, `git-lfs`, bibliotekę `libsndfile` oraz menedżer `uv`, a następnie odtwarza zależności z pliku blokady [6, 7, 9, 22, 2]. Kontener jest przeznaczony przede wszystkim do uruchomień CUDA z użyciem NVIDIA Container Toolkit i z katalogiem danych montowanym z hosta [26]. Nie zastępuje konfiguracji eksperymentu, ale ułatwia odtworzenie środowiska narzędziowego. Repozytorium zawiera testy automatyczne uruchamiane przez `pytest` [29]. Obejmują one spójność domyślnej konfiguracji, parsery argumentów, obliczanie przepływności, obsługę rozmiarów słowników RVQ, pakowanie indeksów, protokół demonstratora oraz kontrolne przejścia przez uproszczone warianty modelu. Testy nie zastępują odsłuchu ani metryk jakości mowy. Ich zadaniem jest wykrywanie regresji w elementach, które łatwo naruszyć podczas zmian architektury i skryptów treningowych.

Kontener i testy pełnią różne funkcje. Kontener ogranicza zmienność środowiska: wersję Pythona, dostępność narzędzi audio, obsługę plików dźwiękowych i integrację z bibliotekami CUDA. Testy sprawdzają natomiast, czy kod zachowuje podstawowe kontrakty po zmianie konfiguracji. W kodeku z RVQ szczególnie łatwo o błąd, który nie powoduje awarii programu, ale zmienia przepływność albo kolejność indeksów. Taki błąd mógłby prowadzić do poprawnie wyglądającego pliku audio, który nie odpowiada już deklarowanemu strumieniowi bitowemu. Najbardziej wrażliwe testy dotyczą obliczania `bitrate'u` i pakowania indeksów. Wzór na przepływność musi obsługiwać zarówno równe słowniki, jak i warianty z różnymi rozmiarami słowników. Pakowanie musi zachować kolejność indeksów w ramce i poprawnie odczytywać szerokość bitową wynikającą z rozmiaru kodownika. Osobno testowany jest protokół demonstratora, ponieważ błąd w nagłówku porcji audio może objawić się dopiero po połączeniu przeglądarki, WebSocketu i backendu inferencyjnego. Testy jednostkowe są tu szybsze niż każdorazowe ręczne uruchamianie całego interfejsu. Ostatnim elementem kontroli jest zgodność między konfiguracją, checkpointem i narzędziem inferencji. Skrypt inferencyjny musi odtworzyć tę samą liczbę słowników, wymiar kodu RVQ, typ aktywacji i sposób zwiększania rozdzielczości czasowej, które były użyte podczas treningu. Niezgodność tych parametrów może dać rekonstrukcję słabej jakości nawet wtedy, gdy plik z wagami zostanie poprawnie załadowany. Dlatego parametry przyjętego wariantu zostały opisane liczbowo w rozdziale, a nie tylko nazwane ścieżką do checkpointu.

Rozdział 4

Eksperymenty i analiza wyników

- 4.1 Opis zbioru danych wykorzystanego w badaniach
- 4.2 Konfiguracja eksperymentów
- 4.3 Wpływ przepływności bitowej na jakość rekonstrukcji
- 4.4 Wpływ parametrów modelu na skuteczność kompresji
- 4.5 Ocena jakości odtworzonego dźwięku
- 4.6 Dyskusja wyników

Rozdział 5

Zakończenie

- 5.1 Wnioski z przeprowadzonych badań
- 5.2 Realizacja celu i zakresu pracy
- 5.3 Ograniczenia techniczne zaproponowanego rozwiązania
- 5.4 Kierunki dalszego rozwoju

Literatura

- [1] Apple Machine Learning Research. MLX documentation. <https://ml-explore.github.io/mlx/build/html/index.html>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [2] Astral Software Inc. uv documentation. <https://docs.astral.sh/uv/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.
- [4] Tom Bäckström and Carl Mägi. Sparse linear prediction for efficient speech coding at low bit rates. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 25(4):789–800, 2017.
- [5] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression, 2022.
- [6] Docker Inc. Dockerfile reference. <https://docs.docker.com/reference/dockerfile/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [7] FFmpeg Developers. FFmpeg documentation. <https://ffmpeg.org/documentation.html>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [8] Philip N. Garner and Miloš Cernak. Sinusoidal coding for low bit-rate speech synthesis. In *Proceedings of Interspeech*, pages 1203–1207, 2019.
- [9] Git Large File Storage Contributors. Git LFS. <https://git-lfs.com/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [11] Robert M. Gray. Vector quantization. *IEEE ASSP Magazine*, 1(2):4–29, 1984.
- [12] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [14] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [15] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhota, Ruslan Salakhutdinov, and Abdelrahman Mohamed. HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.
- [16] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

- [17] Srinivas Kankanahalli. End-to-end optimized speech coding with deep neural networks. In *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 2521–2525, 2018.
- [18] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [19] W. B. Kleijn, F. S. Lim, A. Luebs, J. Skoglund, F. Stimberg, Q. Wang, and T. C. Walters. WaveNet based low rate speech coding. In *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 676–680, 2018.
- [20] Rithesh Kumar, Prem Seetharaman, Alejandro Luebs, Ishaan Kumar, and Kundan Kumar. High-fidelity audio compression with improved RVQGAN. In *Advances in Neural Information Processing Systems*, volume 36, pages 27980–27993, 2023.
- [21] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [22] libsndfile contributors. libsndfile. <https://libsndfile.github.io/libsndfile/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [23] Max Morrison, Justin Xu, and Nicholas J. Bryan. Efficient neural audio codecs for low-latency streaming on edge devices. In *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7862–7866, 2022.
- [24] NATO Standardization Office. STANAG 4591: The NATO 600/1200/2400 bps voice coder. Technical report, NATO Standardization Office, 2015. NATO STANAG.
- [25] NVIDIA Corporation. CUDA c++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [26] NVIDIA Corporation. NVIDIA Container Toolkit documentation. <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [28] Adam Polyak, Yossi Adi, Jade Copet, Eugene Kharitonov, Karan Lakhota, Wei-Ning Hsu, et al. Speech resynthesis from discrete disentangled self-supervised representations. In *Proceedings of Interspeech*, pages 3615–3619, 2021.
- [29] pytest contributors. pytest documentation. <https://docs.pytest.org/en/stable/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [30] python-soundfile contributors. python-soundfile documentation. <https://python-soundfile.readthedocs.io/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [31] PyTorch Contributors. Torchaudio documentation. <https://docs.pytorch.org/audio/stable/>, 2026. Dokumentacja online, dostęp: 2026-06-03.
- [32] Kaizhi Qian, Yang Zhang, Heting Gao, Junrui Ni, Cheng-I Lai, David Cox, Mark Hasegawa-Johnson, and Shiyu Chang. ContentVec: An improved self-supervised speech representation by disentangling speakers. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.
- [33] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

- [34] David Rowe. Codec 2: Open source low bit rate speech codec. In *Proceedings of Linux.conf.au*, 2019.
- [35] David Rowe. Codec 2 at 450 bit/s. Technical report, FreeDV Project, 2021. Technical report.
- [36] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [37] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.
- [38] Jean-Marc Valin, Johannes B  the, and Koen Vos. Real-time ultra-low latency speech enhancement on ARM processors. In *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7867–7871, 2022.
- [39] Jean-Marc Valin and Jan Skoglund. LPCNet: Improving neural speech synthesis through linear prediction. In *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5891–5895, 2019.
- [40] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, et al. WaveNet: A generative model for raw audio, 2016.
- [41] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [43] Ryuichi Yamamoto, Eunwoo Song, and Jae-Min Kim. Parallel WaveGAN: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram. In *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6199–6203, 2020.
- [44] Di Yang, Shuai Liu, Yao Zhou, Hao Zhang, Xing Li, Yao Wang, and Furu Wei. HiFi-Codec: Group-residual vector quantization for high fidelity audio codec, 2023.
- [45] Neil Zeghidour, Aaron Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2021.
- [46] Xiang Zhang, Xu Tan, Tao Qin, Sheng Chen, and Tie-Yan Liu. SpeechTokenizer: Unified speech tokenizer for speech language models, 2023.
- [47] Yutong Zhu and Jean-Marc Valin. Optimizing LPCNet for embedded devices. In *Proceedings of Interspeech*, pages 1831–1835, 2021.
- [48] Liu Ziyin, Tilman Hartwig, and Masahito Ueda. Neural networks fail to learn periodic functions and how to fix it. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Dodatek A

Dodatki

A.1 Repozytorium projektu

A.2 Konfiguracje eksperymentów

A.3 Dodatkowe wyniki i przykłady rekonstrukcji



© 2026 Jakub Aszyk, Kosma Gąsiorowski

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu L^AT_EX na platformie Overleaf.